# Test Cases

AI Alert Correlation & Incident Detection Platform

Generated 2026-10-28 - seed 20261028 - 70 services - 490 raw alerts in scope

## Summary

| TEST TYPE | CASES | PASSED | FAILED | STATUS |
|---|---|---|---|---|
| Positive | 16 | 16 | 0 | ALL PASS |
| Negative | 5 | 5 | 0 | ALL PASS |
| Boundary | 8 | 8 | 0 | ALL PASS |
| Validation | 9 | 9 | 0 | ALL PASS |
| **TOTAL** | **38** | **38** | **0** | ALL PASS |

## Scope and method

Coverage: synthetic data
ranking, evaluation again
designed), Negative (the
integrity).

*How results were obtain
by scripts/generate-test-
(TC-27 to TC-37) could
each case plus a passin*

## A. Dataset generation and ground truth

6 case(s) - 6 pass, 0 fail

| TC-01 \| Positive | PASS |
|---|---|
| FEATURE | Synthetic dataset generation produces a demo-sized alert set |
| TEST INPUT | generateDataset(20261028) — default demo seed |
| EXPECTED RESULT | A dataset with 400-900 raw alerts across the 70 simulated services, spanning a 6-hour window, with at least one ground-truth group per scenario. |
| ACTUAL RESULT | Generated 490 raw alerts, 109 ground-truth groups, 70 services, window 2026-10-28T17:10Z to 2026-10-28T23:10Z. |
| JUSTIFICATION | Confirms the core data pipeline works end to end before anything else runs; the whole demo depends on a realistic alert volume. |

| TC-02 \| Positive | PASS |
|---|---|
| FEATURE | Deterministic generation: same seed reproduces the dataset, a new seed changes it |
| TEST INPUT | generateDataset(20261028) twice, then generateDataset(999999) |
| EXPECTED RESULT | Both runs with the same seed produce byte-identical alerts; a different seed produces a different dataset. |
| ACTUAL RESULT | Same seed identical: true; different seed differs: true (seed A 490 alerts, seed C 500 alerts). |
| JUSTIFICATION | Determinism makes the demo reproducible for judges, while seed variation proves KPIs are computed live and not hardcoded. |

| TC-03 \| Boundary | PASS |
|---|---|
| FEATURE | Alert volume stays inside the 400-900 specification across seeds |
| TEST INPUT | generateDataset over 8 different seeds (20261028, 111222, 987654, 424242, 31337, 555, 771234, 606) |
| EXPECTED RESULT | Every seed yields 400-900 raw alerts, and the counts are not all identical (regression test for the fixed 'same counts every time' bug). |
| ACTUAL RESULT | Counts per seed: 490, 480, 470, 490, 470, 476, 468, 516 (min 468, max 516); varied across seeds: true. |
| JUSTIFICATION | Guards the boundary of the data spec and prevents a regression where the seed no longer influenced dataset size. |

| TC-04 \| Validation | PASS |
|---|---|
| FEATURE | Every alert field is complete and within its valid domain |
| TEST INPUT | Every alert of the default dataset: check service, environment, severity, type, family, numeric value/ threshold, timestamp inside the window, fingerprint format, non-empty description/source |
| EXPECTED RESULT | Zero alerts violate any field rule (enum membership, finite numbers, ts within [windowStart, windowEnd], fingerprint equals env\|service\|type\|metric, known service). |
| ACTUAL RESULT | 0 invalid alerts out of 490 (9 checks each). |
| JUSTIFICATION | A single malformed alert would silently break fingerprinting, correlation grouping and evaluation scoring. |

| TC-05 \| Validation | PASS |
|---|---|
| FEATURE | Ground truth covers every alert exactly once and references real services |
| TEST INPUT | Union of groundTruth[].alertIds vs all alert ids; overlap check between groups; rootCause membership; scenario labels |
| EXPECTED RESULT | Every alert appears in exactly one ground-truth group, no group is empty, every rootCause exists in the service map, and all 5 scripted scenarios are present with their labels. |
| ACTUAL RESULT | 490 alerts covered, 0 uncovered, 0 double-counted, 0 unknown root causes, 0 empty groups, 5/5 scripted scenarios labelled. |
| JUSTIFICATION | Evaluation precision/recall is only meaningful if the answer key is a clean partition of the dataset. |

| TC-06 \| Positive | PASS |
|---|---|
| FEATURE | Correlation engine is deterministic on identical input |
| TEST INPUT | runAnalysis(ds.alerts, DEFAULT_CONFIG) run twice; compare incidents, decisions and evidence (timing fields excluded) |
| EXPECTED RESULT | Both runs produce identical incident grouping, decisions and evidence. |
| ACTUAL RESULT | Two runs identical: true (109 incidents each, undefined decisions each). |
| JUSTIFICATION | Metrics shown on every page must be reproducible for the same dataset, otherwise demo numbers would shift between renders. |

## B. Duplicate suppression (deduplication)

3 case(s) - 3 pass, 0 fail

| TC-07 \| Positive | PASS |
|---|---|
| FEATURE | Duplicate alerts are suppressed and attributed to their canonical alert's incident |
| TEST INPUT | Default dataset: every decision whose alert carries duplicateOf |
| EXPECTED RESULT | At least one duplicate is found; each duplicate references an existing canonical alert, is marked attention=ignore, and is not counted as a unique alert. |
| ACTUAL RESULT | 246 duplicates of 490 raw alerts suppressed -> 244 unique; 0 broken canonical references; 0 duplicates not marked ignore. |
| JUSTIFICATION | Duplicate suppression is the first noise-reduction step: without it, repeated firings would inflate incident counts and mislead the evaluation. |

| TC-08 \| Boundary | PASS |
|---|---|
| FEATURE | Dedup window boundary: exactly 2 minutes is still a duplicate, 2 minutes + 1 s is not |
| TEST INPUT | Two synthetic alerts with identical fingerprint (env\|service\|type\|metric): (a) 120 000 ms apart, (b) 120 001 ms apart; dedupWindowMs = 120 000 |
| EXPECTED RESULT | Case (a): 1 unique + 1 duplicate (window boundary is inclusive). Case (b): 2 unique, 0 duplicates. |
| ACTUAL RESULT | At exactly 120 000 ms: unique=1, duplicates=1. At 120 001 ms: unique=2, duplicates=0. |
| JUSTIFICATION | Off-by-one errors at the dedup boundary would either merge legitimate repeat firings or let duplicates leak into correlation. |

| TC-09 \| Negative | PASS |
|---|---|
| FEATURE | Identical fingerprint outside the dedup window is kept as a separate alert |
| TEST INPUT | Two synthetic alerts, same fingerprint, 600 000 ms (10 min) apart |
| EXPECTED RESULT | 2 unique alerts, 0 duplicates — a repeat firing 10 minutes later is a new event, not a duplicate. |
| ACTUAL RESULT | 10 minutes apart: unique=2, duplicates=0. |
| JUSTIFICATION | Guards the opposite boundary: over-aggressive deduplication would hide genuine recurring problems from the correlation engine. |

## C. Alert correlation engine

8 case(s) - 8 pass, 0 fail

| TC-10 \| Positive | PASS |
|---|---|
| FEATURE | Database exhaustion cascade is grouped into one multi-service incident with the correct root cause |
| TEST INPUT | Default dataset: incident containing primary-db, payment-service and checkout-api |
| EXPECTED RESULT | A single incident contains the DB plus its downstream services, ranks primary-db as probable root cause, carries correlation evidence and a timeline. |
| ACTUAL RESULT | INC-001 "Primary Postgres connection pool exhaustion affecting 6 services" — 6 services, 13 alerts (+6 dup), root cause=primary-db, 17 evidence lines, 14 timeline entries. |
| JUSTIFICATION | This is the product's headline capability: turning many alerts into one actionable incident with an explainable cause. |

| TC-11 \| Positive | PASS |
|---|---|
| FEATURE | All four designed multi-service cascades are detected with the right root causes |
| TEST INPUT | Default dataset: incidents with more than one service |
| EXPECTED RESULT | Exactly 4 multi-service incidents whose probable root causes are primary-db, search-indexer, image-processor and replica-db (scenarios A, B, C, E). |
| ACTUAL RESULT | 4 multi-service incidents; roots = [image-processor, primary-db, replica-db, search-indexer]; expected [image-processor, primary-db, replica-db, search-indexer]. |
| JUSTIFICATION | Proves the engine groups different failure shapes (resource exhaustion, latency propagation, saturation, replication lag) and does not over-merge unrelated noise into them. |

| TC-12 \| Negative | PASS |
|---|---|
| FEATURE | Isolated critical alert (certificate expiry) stays its own incident and is never merged |
| TEST INPUT | Default dataset: the secrets-broker certificate_expiry scenario (1 alert + 1 duplicate) |
| EXPECTED RESULT | The certificate alert forms a single-service incident with severity critical, and secrets-broker is not pulled into any other incident's group by time or dependency rules. |
| ACTUAL RESULT | INC-092: severity=critical, services=1, alerts=1 (+1 dup), root=secrets-broker, leaked into 0 other incidents. |
| JUSTIFICATION | Verifies the engine does not over-correlate: an unrelated critical must remain separate so operators are not sent a misleading story. |

| TC-13 \| Negative | PASS |
|---|---|
| FEATURE | Environment isolation gate blocks a staging alert from joining a production incident |
| TEST INPUT | Synthetic: production primary-db critical connection_pool at t0, then identical-staging alert at t0 + 60 s (same service, inside every other gate) |
| EXPECTED RESULT | The staging alert does NOT join the production incident; its decision records a rejected join with rule 'Environment isolation'. |
| ACTUAL RESULT | Joined across environments: false; gate recorded on decision: true; resulting incidents: 2. |
| JUSTIFICATION | Cross-environment merging would create false incidents between staging and production — a classic false positive for correlation engines. |

| TC-14 \| Boundary | PASS |
|---|---|
| FEATURE | Temporal proximity gate: 180 s joins a cascade, 181 s does not |
| TEST INPUT | Synthetic: primary-db critical resource alert at t0; payment-service critical error alert (1 dependency hop) at t0 + 180 000 ms, then repeated at t0 + 181 000 ms. temporalGateMs = 180 000 |
| EXPECTED RESULT | Exactly 180 s: both alerts share one incident (score 5 = threshold). 181 s: separate incidents, decision shows a 'Temporal proximity' gate rejection. |
| ACTUAL RESULT | Gap 180 s -> 1 incident(s) (joined: true); gap 181 s -> 2 incidents (split: true); temporal gate recorded on the late alert: true. |
| JUSTIFICATION | The temporal gate is what separates a real cascade from coincidental alerts; its boundary must be exact and inspectable. |

| TC-15 \| Boundary | PASS |
|---|---|
| FEATURE | Join score threshold: 1-hop dependency reaches the threshold, 2-hop does not |
| TEST INPUT | Synthetic pairs at 60 s gap, identical severities, different alert families: service pair at dependency distance 1 vs distance 2; joinThreshold = 5 (hop points 3 vs 2, temporal 2) |
| EXPECTED RESULT | Distance 1: score 5 -> joined. Distance 2: score 4 -> rejected with 'No incident reached the score threshold'. |
| ACTUAL RESULT | 1-hop pair (web-bff -> mobile-bff, distance 1) joined: true; 2-hop pair (web-bff -> websocket-gateway, distance 2) joined: false; rejection reasons: Dependency link. |
| JUSTIFICATION | The scoring threshold is the difference between joining a real cascade and gluing unrelated alerts together; both sides of the boundary must behave as designed. |

| TC-16 \| Boundary | PASS |
|---|---|
| FEATURE | Incident time window: same-service alert at exactly 15 min joins, 15 min + 1 s is rejected |
| TEST INPUT | Synthetic: primary-db alert at t0 and a second primary-db alert (different metric) at t0 + 900 000 ms, then t0 + 900 001 ms; timeWindowMs = 900 000 |
| EXPECTED RESULT | Exactly 15 min -> joined (same service, score 6). Beyond -> separate incident with a 'Time window' gate rejection. |
| ACTUAL RESULT | At exactly 15 min -> joined: true; at 15 min + 1 s -> joined: false, time-window gate recorded: true. |
| JUSTIFICATION | The time window bounds how long one incident can keep absorbing alerts; boundary errors would either stretch incidents indefinitely or cut them mid-cascade. |

| TC-17 \| Validation | PASS |
|---|---|
| FEATURE | Every raw alert receives exactly one decision with a reason |
| TEST INPUT | All decisions produced by runAnalysis on the default dataset |
| EXPECTED RESULT | decision count == raw alert count; every decision has a valid attention value and a non-empty reason explaining inclusion, rejection or suppression. |
| ACTUAL RESULT | 490 decisions for 490 alerts; 0 missing reasons; 0 invalid attention values; 0 without rule evidence. |
| JUSTIFICATION | The product promises that every correlation decision is inspectable; an alert without a decision would be invisible in the UI. |

## D. Attention triage (act / review / ignore)

2 case(s) - 2 pass, 0 fail

| TC-18 \| Positive | PASS |
|---|---|
| FEATURE | Triage rules: critical alerts demand action, suppressed duplicates are ignored |
| TEST INPUT | All decisions of the default dataset |
| EXPECTED RESULT | Every non-duplicate critical alert is attention=action; every duplicate is attention=ignore; no alert receives an unknown label. |
| ACTUAL RESULT | 0 critical alerts not marked action; 0 duplicates not marked ignore (of 246 duplicates). |
| JUSTIFICATION | The demo moment is 'act on these, ignore those' — wrong labels would tell an operator to ignore a critical outage. |

| TC-19 \| Boundary | PASS |
|---|---|
| FEATURE | Attention distribution reflects a noisy system: most alerts are ignorable, few need action |
| TEST INPUT | Counts of action / review / ignore across all decisions |
| EXPECTED RESULT | All three buckets are non-empty, they sum to the raw alert count, ignore > review > action (the noise-reduction story). |
| ACTUAL RESULT | action=9, review=49, ignore=432 (sum 490 of 490 alerts); ordered as expected: true. |
| JUSTIFICATION | Validates the core UX promise: hundreds of alerts collapse to a handful of page-worthy signals without hiding real incidents. |

## E. Root-cause ranking

2 case(s) - 2 pass, 0 fail

| TC-20 \| Positive | PASS |
|---|---|
| FEATURE | Root cause top-1 matches the labelled root cause for all 5 scenarios |
| TEST INPUT | For each scripted ground-truth group: best-overlap predicted incident vs its labelled rootCause |
| EXPECTED RESULT | 5 of 5 scenarios rank the correct service first (top-1 accuracy 1.0). |
| ACTUAL RESULT | 5/5 correct. GT-1: predicted primary-db vs labelled primary-db -> OK; GT-2: predicted search-indexer vs labelled search-indexer -> OK; GT-3: predicted image-processor vs labelled image-processor -> OK; GT-4: predicted secrets-broker vs labelled secrets-broker -> OK; GT-5: predicted replica-db vs labelled replica-db -> OK. |
| JUSTIFICATION | Root-cause accuracy is the headline metric on the Evaluation page; a wrong ranking would undermine every explanation the product gives. |

| TC-21 \| Validation | PASS |
|---|---|
| FEATURE | Root-cause candidates are well formed: sorted, evidenced, and drawn from the incident |
| TEST INPUT | rootCauses[] of every incident in the default analysis |
| EXPECTED RESULT | Candidates are sorted by descending score, each carries at least one signal with points, every score >= 0, and probableRootCause is a member of the incident's services. |
| ACTUAL RESULT | 0 incidents with malformed candidate lists (of 109). |
| JUSTIFICATION | The UI renders ranked candidates with evidence; malformed rankings would show nonsense to a judge mid-demo. |

## F. Evaluation and baseline comparison

4 case(s) - 4 pass, 0 fail

| TC-22 \| Positive | PASS |
|---|---|
| FEATURE | Correlation engine outperforms the naive baseline on grouping quality |
| TEST INPUT | evaluate(): baseline = one incident per unique alert; proposed = correlation engine; Jaccard >= 0.5 matching against ground truth |
| EXPECTED RESULT | Proposed precision and recall both 1.0 with 0 false positives/negatives, and proposed F1 strictly greater than baseline F1. |
| ACTUAL RESULT | Engine: P=1.00, R=1.00, F1=1.00, FP=0, FN=0. Baseline: P=0.43, R=0.96, F1=0.59, FP=139, FN=4. |
| JUSTIFICATION | This is the measurable claim of the whole project: correlation reduces noise without losing or inventing incidents. |

| TC-23 \| Positive | PASS |
|---|---|
| FEATURE | Baseline detector emits exactly one incident per unique alert |
| TEST INPUT | evaluate(): baselineIncidentCount vs uniqueAlertCount |
| EXPECTED RESULT | Baseline incident count equals the unique alert count (its stated definition), and alert-reduction rate for the baseline is 0. |
| ACTUAL RESULT | Baseline incidents=244, unique alerts=244, baseline reduction=0.00, engine incidents=109 (engine reduction=0.55). |
| JUSTIFICATION | Anchors the comparison: the baseline is defined precisely, so the engine's reduction figure means something. |

| TC-24 \| Boundary | PASS |
|---|---|
| FEATURE | Held-out dataset evaluates cleanly on an unseen seed |
| TEST INPUT | generateDataset(${HOLDOUT}) (seed + 977) -> runAnalysis -> evaluate |
| EXPECTED RESULT | No exception; ground-truth groups exist; all precision/recall/F1 values within [0,1]; predicted and ground-truth counts positive. |
| ACTUAL RESULT | Held-out seed 20262005: 452 alerts, GT=102, predicted=102, engine P=1.00 R=1.00, baseline F1=0.59. |
| JUSTIFICATION | Proves the engine and evaluator generalise beyond the demo seed and are not tuned to one dataset. |

| TC-25 \| Validation | PASS |
|---|---|
| FEATURE | Evaluation output is complete and internally consistent |
| TEST INPUT | EvaluationResult of the default seed: rows, metric ranges, matching description, match detail |
| EXPECTED RESULT | 12 metric rows each with label and definition; all precision/recall/F1 values in [0,1]; matching-method text mentions the Jaccard rule; one match-detail row per ground-truth group. |
| ACTUAL RESULT | rows=12/12, values in range=true, method text present=true, match detail rows=109 vs GT 109. |
| JUSTIFICATION | The Evaluation page renders these structures directly; missing or out-of-range values would show blank or absurd figures to judges. |

## G. AI explanation layer

2 case(s) - 2 pass, 0 fail

| TC-26 \| Positive | PASS |
|---|---|
| FEATURE | Deterministic explanation fallback produces a complete, evidence-scoped narrative |
| TEST INPUT | explainIncident(cascade incident, dataset alerts, 'deterministic', degraded=true) — the path used when no AI key is configured |
| EXPECTED RESULT | Summary >= 50 chars; at least 2 competing hypotheses where every named service is either a member of the incident or an engine-ranked candidate for it (never an invented service); each hypothesis carries supporting evidence and a non-increasing score order; at least 3 investigation steps; generator labelled 'deterministic'. |
| ACTUAL RESULT | Summary 410 chars; 3 hypotheses [image-processor (member), thumbnailer (member), asset-cdn (ranked candidate)] — all engine-backed: true, all with evidence: true, ranked order: true; 5 steps; generator=deterministic, degraded=true. |
| JUSTIFICATION | The AI feature must never be a single point of failure: with no API key the demo still needs a credible explanation, and that explanation must stay inside the engine's own evidence (candidates are incident members plus their ranked upstream dependencies by design). |

| TC-27 \| Validation | PASS |
|---|---|
| FEATURE | AI output is sanitised: model may only restate incident evidence and falls back on failure |
| TEST INPUT | Static review of src/lib/store.tsx (explain flow) and src/convex/ai.ts (server action): hypotheses filtered by `allowed = new Set(incident.services)`, JSON parse guarded by try/catch -> fallback(), 9 s timeout race, server returns ok:false when no key |
| EXPECTED RESULT | Model hypotheses naming a service outside the incident are dropped; malformed JSON or a failed/timed-out call triggers the deterministic fallback; keys are read only from Convex environment variables. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | Prevents the LLM from fabricating evidence or merging incidents, and guarantees the app stays fully functional when no API key exists. |

## H. User interface and integration

11 case(s) - 11 pass, 0 fail

| TC-28 \| Positive | PASS |
|---|---|
| FEATURE | Run analysis button re-runs detection and refreshes the status indicator |
| TEST INPUT | Static review of src/components/AppShell.tsx handleRun -> store.runAnalysisNow (sets config signature, bumps runNonce, logs activity); status chip renders incident count and duration, or 'stale — re-run analysis' when the dataset changed |
| EXPECTED RESULT | Clicking Run analysis recomputes incidents, the button shows a running state for the duration, and the indicator returns to the fresh 'N incidents · Nms' text. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | The primary control of the demo must visibly do something; judges watch this button to confirm the numbers are live. |

| TC-29 \| Positive | PASS |
|---|---|
| FEATURE | Live Alerts search and filters accept valid input and narrow the table |
| TEST INPUT | Static review of src/pages/LiveAlerts.tsx filter chain: trim/lowercase search over alert id + service, severity/service/type/attention Selects, time range 15m/1h/3h/6h/all; empty search string bypasses the search predicate |
| EXPECTED RESULT | Empty search shows all in-scope alerts; a partial id or service name (case-insensitive) matches only related rows; each Select further narrows the result set; filters combine with AND. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | The alert explorer is the v1 centrepiece — filters are how a judge finds the one alert they care about among hundreds. |

| TC-30 \| Boundary | PASS |
|---|---|
| FEATURE | Pagination resets on filter change and clamps out-of-range pages |
| TEST INPUT | Static review of src/pages/LiveAlerts.tsx: setPage(0) on every filter/search change (9 call sites) and `currentPage = Math.min(page, pageCount - 1)` |
| EXPECTED RESULT | Changing a filter always returns to page 1; if results shrink below the current page the view clamps to the last available page instead of rendering a blank table. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | Classic pagination boundary bug: filtering on a later page can leave the user staring at an empty screen with no explanation. |

| TC-31 \| Validation | PASS |
|---|---|
| FEATURE | Empty filter results show an explicit message instead of a blank area |
| TEST INPUT | Static review of empty states: LiveAlerts 'No alerts match the current filters.', Incidents 'No incidents match these filters.', RootCause queue 'No incidents in this run...', ServiceMap 'No incidents for this service.' |
| EXPECTED RESULT | Every list view renders a human-readable empty-state message when its filter yields zero rows. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | A blank table reads as a broken app during a demo; an explicit message tells the user what to change. |

| TC-32 \| Positive | PASS |
|---|---|
| FEATURE | Incident status controls update application state and downstream views |
| TEST INPUT | Static review of src/lib/store.tsx setStatus (writes statuses map + activity log) and consumers: incidents memo overrides status, ServiceMap excludes resolved from 'active', Overview KPI counts non-resolved |
| EXPECTED RESULT | Acknowledging/resolving/reopening an incident immediately changes its badge, removes it from active counts when resolved, and re-opens it on reopen; the action is logged in the activity feed. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | Status controls are required for the investigation workflow; state that does not propagate would make the demo feel fake. |

| TC-33 \| Negative | PASS |
|---|---|
| FEATURE | Unknown incident id renders a not-found panel with a recovery link |
| TEST INPUT | Static review of src/pages/IncidentDetail.tsx: `if (!incident)` early return renders 'Incident not found' with a link back to /dashboard/incidents (route param incidentId from useParams) |
| EXPECTED RESULT | Navigating to /dashboard/incidents/DOES-NOT-EXIST shows the not-found panel and a working return link, never a crash or an empty page. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | URLs are shared and typed; a bad id must degrade gracefully instead of white-screening mid-presentation. |

| TC-34 \| Validation | PASS |
|---|---|
| FEATURE | Auth form rejects empty and malformed email input and reports OTP errors |
| TEST INPUT | Static review of src/pages/Auth.tsx: email Input has type="email" and required; handleEmailSubmit/ handleOtpSubmit wrap signIn in try/catch and render `error` text below the field |
| EXPECTED RESULT | Submitting with an empty field is blocked by the browser (required), a malformed address by type=email, and a wrong verification code shows 'The verification code you entered is incorrect.' without leaving the page. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | Form validation is the first line of defence against bad input; silent failures would strand users on the sign-in screen. |

| TC-35 \| Validation | PASS |
|---|---|
| FEATURE | returnTo redirect parameter cannot escape the site (open-redirect guard) |
| TEST INPUT | Static review of resolveRedirectAfterAuth in src/pages/Auth.tsx: accepts the value only when it starts with '/' and not '//'; anything else falls back to /dashboard |
| EXPECTED RESULT | returnTo=/dashboard/alerts returns the user to the alerts page after sign-in; returnTo=//evil.example or https://evil.example falls back to the safe default. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | An unvalidated redirect lets an attacker turn a legitimate sign-in page into a phishing hop. |

| TC-36 \| Positive | PASS |
|---|---|
| FEATURE | All six sidebar destinations exist and unknown URLs hit the 404 page |
| TEST INPUT | Static review of src/main.tsx route table (index, alerts, incidents, incidents/:id, services, rca, evaluation under /dashboard, plus '*' -> NotFound) and AppShell NAV_ITEMS links |
| EXPECTED RESULT | Every sidebar item resolves to a mounted route; an unknown path renders the NotFound page; no nav link points at a missing route. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | Broken navigation is the most visible kind of dead control — one dead link undermines trust in the whole demo. |

| TC-37 \| Negative | PASS |
|---|---|
| FEATURE | Environment selector is removed from the top bar (decluttering regression) |
| TEST INPUT | Static review of src/components/AppShell.tsx: no Select import and no environment control in the header; header contains status indicator + Run analysis only |
| EXPECTED RESULT | The top bar shows exactly the simulation status and the Run analysis button; no production/staging/development dropdown anywhere in the chrome. |
| ACTUAL RESULT | Code-path review confirms the expected behaviour is implemented as described (the exact source locations are listed under Test input), and 'bunx tsc -b --noEmit' passes for this code. No browser runtime is available in this environment, so this is a static verification rather than a runtime observation. |
| JUSTIFICATION | Locks in the requested scope change so a future edit does not silently reintroduce the clutter. |

| TC-38 \| Positive | PASS |
|---|---|
| FEATURE | Project passes the TypeScript build check |
| TEST INPUT | bunx tsc -b --noEmit (project root), same command the platform runs after every change |
| EXPECTED RESULT | Exit code 0 with no type errors across the app, engine, store and Convex sources. |
| ACTUAL RESULT | bunx tsc -b --noEmit exited 0 in 10.3s — no type errors. |
| JUSTIFICATION | A type error blocks the platform check and can white-screen the preview; this is the gate for every other test being meaningful. |

## How to reproduce

Run `bun scripts/generate-test-pdf.ts` from the project root. The script executes every engine test case against the current source, prints PASS/FAIL per case, and rewrites this PDF. Typecheck separately with `bunx tsc -b --noEmit`; Convex functions compile with `bun convex dev --once`.

*All data in this product is synthetic and generated client-side; no real monitoring infrastructure, IPs or customer identifiers are involved. AI explanations are optional: without an API key the deterministic generator is used and labelled as such.*